

Turbos Clmm Math

Audit Report

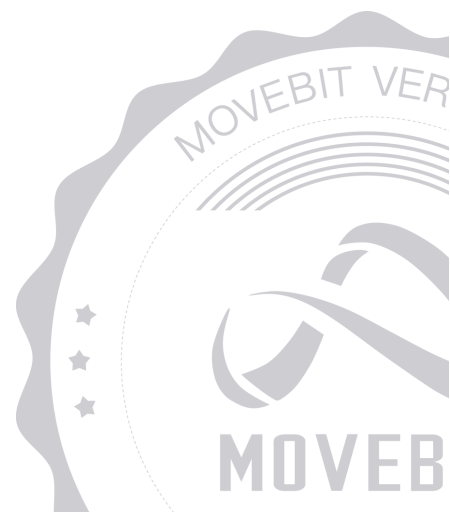


contact@bitslab.xyz



https://twitter.com/movebit_

Sat May 24 2025



Turbos Clmm Math Audit Report

1 Executive Summary

1.1 Project Information

Description	DEX
Type	Dex
Auditors	MoveBit
Timeline	Sat May 24 2025 - Sat May 24 2025
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/turbos-finance/turbos-clmm
Commits	66eccab8e5adc902e61b1574475c7eedd50dbfee1cd4d973d014ad64fd5cbd079f19f351d777a9fd

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
FMU3	sources/lib/full_math_u32.move	49da8c7f42cdb7d5ee47e2fc3abc82fa9c51b7dc
MU2	sources/lib/math_u256.move	524382de0285f2392cdf268cad9e33f7061afca7
MU1	sources/lib/math_u128.move	3bb58e66d3d2a40debb1acb503793188eabc1465
MLI	sources/lib/math_liquidity.move	d94988b2d7cfe2e5cf026ea89838371378e708c0
FMU6	sources/lib/full_math_u64.move	be9e7b673a23a10242949077c84cf6b888de380b
MBI	sources/lib/math_bit.move	d409aef76da297737d69c4d2771c9dda1aaaaa3d
FMU1	sources/lib/full_math_u128.move	f37cedac47ecdbbd3c11bb684ae525648b8ac19e
STO	sources/lib/string_tools.move	06f713f7fa6c553e7aa5c61c86bc5797aedb4a94
I32	sources/lib/i32.move	343db86e3cd3d0c18f05b87d4bb63f0e5e223981
I12	sources/lib/i128.move	364951c270ef74ded0522c68af78dfb2ba4e1622

I64	sources/lib/i64.move	90887a84d88c32a1eb4db1768a25436e1017a4ea
MSP	sources/lib/math_sqrt_price.move	4a4d77c1a60cc71f8b32182600f693dffec19ce5
MSW	sources/lib/math_swap.move	8b3ff2152d377080735459e39bf9fe085628e9fc
MU6	sources/lib/math_u64.move	e025fd7485f1381daa8efcb31d54101ccc719ac8
MTI	sources/lib/math_tick.move	4d32199cba6c59a6fd5994e5301e3b261a0d381c
MU2	sources/lib/math_u256.move	d87ab62cade111c4b02fe7458f3fe41470946e79
MU1	sources/lib/math_u128.move	64329496dddde2beb4423eb0a5e93b9b7df85daaf
MLI	sources/lib/math_liquidity.move	2ef3f817e856496b3ec245dbe53b04358eff1758
MSP	sources/lib/math_sqrt_price.move	f4dd522e2fd668d1d9ba6826b89369961dab94d1

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	4	4	0
Informational	0	0	0
Minor	4	4	0
Medium	0	0	0
Major	0	0	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [Turbos](#) to identify any potential issues and vulnerabilities in the source code of the [Turbos Clmm Math](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 4 issues of varying severity, listed below.

ID	Title	Severity	Status
MLI-1	Potential Overflow Risks Leading to Numerical Calculation Errors	Minor	Fixed
MSP-1	Asset Overpricing and Liquidity Overflow Risks	Minor	Fixed
MU1-1	Missing Overflow Check in Arithmetic	Minor	Fixed
MU2-1	Overflow Risk Due to Incorrect Boundary Check	Minor	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the [Turbos Clmm Math](#) Smart Contract :

This project is clmm's mathematical infrastructure, encapsulating various mathematical calculations, including most of the calculations of clmm's mathematical logic, and the calculations of some common data types: unsigned integer, signed integer, tick maths, liquidity maths, swap maths, sqrt_price maths and so on.

4 Findings

MLI-1 Potential Overflow Risks Leading to Numerical Calculation Errors

Severity: Minor

Status: Fixed

Code Location:

sources/lib/math_liquidity.move#98-110

Descriptions:

Let's look at the logic of the code that creates the problem.

```
public fun get_amount_a_for_liquidity(
  sqrt_price_a: u128,
  sqrt_price_b: u128,
  liquidity: u128
): u128 {
  if (sqrt_price_a > sqrt_price_b) (sqrt_price_a, sqrt_price_b) = (sqrt_price_b,
sqrt_price_a);

  full_math_u128::mul_div_floor(
    liquidity << RESOLUTION, // @audit may be overflow
    sqrt_price_b - sqrt_price_a,
    sqrt_price_b
  ) / sqrt_price_a
}
```

At this time, the liquidity will be shifted, if the liquidity amount is very large will cause overflow problems, resulting in incorrect calculation results, but this function is not called in the core business logic of clmm, so there will be no loss of funds.

Suggestion:

Add an overflow check here to prevent the shift from overflowing.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

MSP-1 Asset Overpricing and Liquidity Overflow Risks

Severity: Minor

Status: Fixed

Code Location:

`sources/lib/math_sqrt_price.move#172-188`

Descriptions:

We found that the maximum value of the price 79226673515401279992447579055 is 96 bits, and because the maximum liquidity is a u128 data type, there is a risk of shift overflow in the following calculation.

```
fun get_next_sqrt_price_from_amount_a_rounding_up(
  sqrt_price: u128,
  liquidity: u128,
  amount: u128,
  add: bool
): u128 {
  if (amount == 0) return sqrt_price;
  let (sqrt_price_u256, liquidity_u256, amount_u256) = ((sqrt_price as u256), (liquidity as u256), (amount as u256));

  let p = amount_u256 * sqrt_price_u256;
  let numerator = (liquidity_u256 * sqrt_price_u256) << RESOLUTION; //@audit may be overflow
  //todo check numerator overflow u256
  let liquidity_shl = liquidity_u256 << RESOLUTION;
  let denominator = if (add) liquidity_shl + p else liquidity_shl - p;

  (math_u256::div_round(numerator, denominator, true) as u128)
}
```

But this function is also used for capital calculations, so there is no risk of capital loss.

Suggestion:

It is recommended that checks be added to prevent overflow issues.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

MU1-1 Missing Overflow Check in Arithmetic

Severity: Minor

Status: Fixed

Code Location:

`sources/lib/math_u128.move;`

`sources/lib/math_u256.move;`

`sources/lib/math_u64.move`

Descriptions:

In the previous implementation of the `wrapping_add()` function, the return value indicating overflow (o) was not properly handled — the result was used directly without checking whether an overflow had occurred. This effectively silently allowed arithmetic overflows, as the calculation would wrap around if the result exceeded the u128 upper bound.

In practical deployments, overflow is unlikely to occur under normal operating conditions.

The data involved in the addition are almost impossible above the u128.

Suggestion:

Now, if any addition exceeds the u128 range, the transaction fails gracefully with a meaningful error (EOverflow) rather than continuing with invalid data.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

MU2-1 Overflow Risk Due to Incorrect Boundary Check

Severity: Minor

Status: Fixed

Code Location:

sources/lib/math_u256.move#19

Descriptions:

The function `checked_shlw(n: u256): (u256, bool)` is intended to safely shift a `u256` value `n` left by 64 bits, and return a flag if the operation would cause an overflow. The function uses the following logic to check for overflow:

```
let mask = 1 << 192;
if (n > mask) {
  (0, true)
} else {
  ((n << 64), false)
}
```

However, the overflow check is flawed. Specifically, when `n == 2192`, the shift operation `n << 64` results in `2256`, which exceeds the maximum value representable by a `u256` (`2256 - 1`). Because the current condition uses `n > 2192`, the value `2192` itself is allowed to proceed into the `else` block, which leads to an overflow.

Suggestion:

Update the conditional check from:

```
if (n > 1 << 192)
```

to:

```
if (n >= 1 << 192)
```

This ensures that any value which would cause an overflow when shifted is properly caught and handled.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

